



# *Ingeniería Aumentada con Sistemas Inteligentes*

Volumen III

Del método a la Metodología





# Tabla de contenidos

|             |                                       |           |
|-------------|---------------------------------------|-----------|
| .1.         | Metodología IASI                      | 5         |
|             | Del método a la metodología           | 7         |
|             | Cómo leer este volumen                | 9         |
| <b>I.</b>   | <b>Fundamentos de IASI</b>            | <b>11</b> |
| I.1.        | Qué es IASI                           | 15        |
| I.2.        | Principios metodológicos              | 17        |
| I.3.        | Autoridad y fuentes de verdad         | 19        |
| I.4.        | Artefactos y trazabilidad             | 21        |
| <b>II.</b>  | <b>La metodología</b>                 | <b>23</b> |
| II.1.       | Inputs                                | 27        |
| II.2.       | Definitions                           | 35        |
| II.3.       | Rules y configuración                 | 37        |
| II.4.       | Specifications                        | 39        |
| II.5.       | Tasks                                 | 41        |
| II.6.       | Validations                           | 43        |
| II.7.       | Estados y aceptación                  | 45        |
| II.8.       | Cambio e iteración                    | 47        |
| <b>III.</b> | <b>Estructura de un proyecto IASI</b> | <b>49</b> |
| III.1.      | La raíz del proyecto                  | 53        |
| III.2.      | Estructura de inputs                  | 55        |
| III.3.      | Artefactos de ingeniería              | 57        |
| III.4.      | Estado local de IASI                  | 59        |
| III.5.      | Directorios derivados de adapters     | 61        |
| III.6.      | Outputs y releases                    | 63        |
| <b>IV.</b>  | <b>Workflows</b>                      | <b>65</b> |
| IV.1.       | Modelo de workflow                    | 69        |
| IV.2.       | De inputs a definitions               | 71        |
| IV.3.       | Planificación y ejecución             | 73        |
| IV.4.       | Validación y aceptación               | 75        |
| IV.5.       | Workflow de cambio                    | 77        |
| <b>V.</b>   | <b>Commands</b>                       | <b>79</b> |
| V.1.        | Modelo de command                     | 83        |
| V.2.        | validate                              | 85        |
| V.3.        | Commands de task                      | 87        |
| V.4.        | archive                               | 89        |
| V.5.        | adapter                               | 91        |

|                                                |            |
|------------------------------------------------|------------|
| <b>VI. Adapters e integración</b>              | <b>93</b>  |
| VI.1. Por qué existen los adapters             | 97         |
| VI.2. Canónico y derivado                      | 99         |
| VI.3. Adapters de agente                       | 101        |
| VI.4. Agente, modelo y runtime                 | 103        |
| VI.5. Capacidades y degradación                | 105        |
| <b>VII. Mecanismos transversales</b>           | <b>107</b> |
| VII.1. Rules                                   | 111        |
| VII.2. Skills                                  | 113        |
| VII.3. Templates                               | 115        |
| VII.4. Configuration                           | 117        |
| VII.5. Automations                             | 119        |
| VII.6. Ámbito y precedencia                    | 121        |
| VII.7. Contenido derivado y contenido de autor | 123        |
| Referencias                                    | 125        |
| Licencias                                      | 127        |

# .1. Metodología IASI

Volumen III · Ingeniería Aumentada con Sistemas Inteligentes





## Del método a la metodología

IASI nació de una pregunta práctica: **¿cómo hacemos ingeniería cuando un Sistema Inteligente participa en el propio trabajo de ingeniería?**

Responderla exige algo más que aprender a utilizar un modelo, un agente o una interfaz.

Necesitamos saber qué información tiene autoridad, cómo se convierte en conocimiento operativo, qué trabajo puede ejecutarse, cómo se valida, qué estado puede aceptarse y qué ocurre cuando cambian las fuentes de las que dependía una decisión.

Este volumen describe esa metodología.

No presenta una receta ligada a una herramienta concreta. Define un sistema de artefactos, responsabilidades, ciclos de vida, workflows, commands y adapters que permite trabajar sobre agentes y modelos diferentes conservando una semántica común.

La idea que atraviesa todo el volumen puede resumirse así:

|             |                                          |
|-------------|------------------------------------------|
| metodología | → qué debe ocurrir                       |
| estructura  | → dónde vive                             |
| workflow    | → cómo se recorre                        |
| command     | → cómo se solicita                       |
| adapter     | → cómo se ejecuta en un entorno concreto |

La herramienta `iasi` materializa y automatiza partes de ese sistema.

Pero la herramienta no es la metodología.

Y el agente tampoco.



## Cómo leer este volumen

Este volumen tiene dos naturalezas.

Por una parte, desarrolla una metodología. Conviene leer sus primeras partes en orden porque cada concepto prepara el siguiente: autoridad, artefactos, inputs, definitions, specifications, tasks, validations y estados forman un sistema.

Por otra, es también un documento de referencia. Las partes dedicadas a estructura, workflows, commands y adapters están pensadas para volver a ellas durante el trabajo cotidiano.

La terminología se mantiene deliberadamente estable. Cuando IASI utiliza palabras como **input**, **definition**, **specification**, **task**, **validation**, **state**, **workflow**, **command** o **adapter**, no las utiliza como sinónimos informales. Cada una representa una función distinta dentro del método.

El objetivo no es memorizar una jerarquía de carpetas o una colección de órdenes.

El objetivo es comprender **qué información posee autoridad, qué puede derivarse de ella, quién puede cambiarla, cómo se comprueba un resultado y cómo se conserva la historia de esas decisiones.**



**Parte I.**

# **Fundamentos de IASI**



Una metodología no comienza por sus comandos.

Comienza por las decisiones que determinan **qué consideramos conocimiento, qué consideramos evidencia, quién tiene autoridad para decidir y cómo sabemos que un resultado puede aceptarse.**

IASI se apoya en un conjunto pequeño de principios que permanecen estables aunque cambien los modelos, los agentes, los lenguajes o las herramientas utilizadas para ejecutar el trabajo.

Esta parte establece esos principios antes de describir el ciclo metodológico.



## I.1. Qué es IASI

IASI es una metodología de **Ingeniería Aumentada con Sistemas Inteligentes**.

Su propósito es permitir que personas y Sistemas Inteligentes participen en un mismo proceso de ingeniería sin convertir una conversación, un prompt o el comportamiento particular de una herramienta en la fuente de verdad del proyecto.

IASI define:

- artefactos con semántica explícita;
- relaciones entre esos artefactos;
- reglas de autoridad y precedencia;
- ciclos de vida y estados;
- mecanismos de validación;
- workflows que recorren el proceso;
- commands que solicitan operaciones canónicas;
- adapters que proyectan esas operaciones sobre entornos concretos.

IASI **no es un modelo de lenguaje**.

Tampoco es un agente central que gobierna el proyecto.

Un agente puede analizar, proponer, transformar, ejecutar o validar trabajo. Pero su autoridad no procede de ser «el agente de IASI». La autoridad permanece en las fuentes, los artefactos canónicos, las decisiones aprobadas y las responsabilidades humanas.

```
fuentes de autoridad
    ↓
artefactos de ingeniería
    ↓
workflows y commands
    ↓
agentes · herramientas · personas
```

Esto permite utilizar IASI sobre entornos diferentes sin redefinir la metodología para cada uno.

La implementación **iasi** puede automatizar operaciones, comprobar estructura, materializar adapters o gestionar estados. Pero **la herramienta implementa la metodología; no la sustituye**.



## I.2. Principios metodológicos

IASI parte de una idea sencilla: una conversación es excelente para pensar, pero es un lugar frágil para conservar el conocimiento de ingeniería.

Una decisión importante puede escribirse en un chat o en un documento con un coste prácticamente idéntico. En el segundo caso, además, puede quedar versionada, revisada, trazada, reutilizada y procesada por herramientas.

Por eso IASI intenta que el conocimiento relevante **termine materializado en artefactos persistentes**.

De este principio se derivan otros.

### I.2.1. El prompt no es la fuente de verdad

El prompt puede iniciar una acción y aportar contexto transitorio. No debe convertirse en el único lugar donde viven requisitos, decisiones, restricciones o criterios de aceptación.

### I.2.2. La evidencia y la interpretación se separan

Lo recibido no se reescribe para hacerlo más cómodo.

Especialmente, `inputs/externals/` conserva exactamente aquello que llegó desde fuera. La normalización y la interpretación producen nuevos artefactos.

### I.2.3. No se actúa más allá de lo conocido

La falta de información no obliga siempre a detener todo el trabajo. Limita el alcance de aquello que puede hacerse legítimamente.

IASI puede avanzar hasta donde permitan la información disponible, las restricciones y el alcance explícito. Lo que no puede hacer es completar silenciosamente los huecos mediante invención.

### I.2.4. Lo derivado no sustituye a su fuente

Código, documentos, configuraciones, adapters y otros resultados pueden ser materializados a partir de artefactos anteriores.

Cuando eso ocurre debe conservarse la relación entre fuente y derivado.

### I.2.5. La validación es parte del proceso

La generación de un resultado no demuestra que sea correcto.

IASI separa ejecución y validación y conserva evidencia suficiente para justificar el estado alcanzado.

### I.2.6. Las herramientas son sustituibles

Un command de IASI debe conservar su significado aunque cambie el agente que lo ejecuta.

Los adapters absorben las particularidades del entorno; la metodología permanece.

### **I.2.7. La responsabilidad no se delega al Sistema Inteligente**

El Sistema Inteligente puede aumentar la capacidad de quien trabaja con él.

La decisión profesional, la autoridad y la responsabilidad siguen perteneciendo a las personas y roles que las tienen asignadas.

## I.3. Autoridad y fuentes de verdad

Uno de los problemas que aparecen al incorporar Sistemas Inteligentes a la ingeniería es confundir **capacidad de actuación** con **autoridad**.

Un agente puede ser capaz de modificar cien archivos. Eso no significa que pueda decidir legítimamente qué debe contener el sistema.

IASI distribuye la autoridad entre artefactos con responsabilidades diferentes:

|                             |                                               |
|-----------------------------|-----------------------------------------------|
| <code>inputs</code>         | → de dónde procede la información             |
| <code>definitions</code>    | → qué hemos comprendido                       |
| <code>rules</code>          | → qué normas deben respetarse                 |
| <code>configuration</code>  | → con qué parámetros trabajamos               |
| <code>specifications</code> | → qué resultado se pretende obtener           |
| <code>tasks</code>          | → qué trabajo concreto debe ejecutarse        |
| <code>validations</code>    | → qué evidencia demuestra el resultado        |
| <code>states</code>         | → cuál es el estado materializado del proceso |

Ninguna de estas piezas debe interpretarse aisladamente.

Una **specification** no puede contradecir una **rule** aplicable. Una **task** no debe inventar alcance que no exista en su **specification**. Un estado **done** carece de valor si no puede relacionarse con el trabajo y las validaciones que lo justifican.

La precedencia tampoco se reduce a «lo último gana».

Cada clase de artefacto posee sus propias reglas. En el caso de las reglas, por ejemplo, una regla más específica puede prevalecer sobre una genérica cuando ambas regulan el mismo aspecto. En estructuras heredables, el ámbito más próximo puede refinar el superior.

La idea común es:

**La autoridad debe ser explícita y resoluble.**

Cuando dos fuentes incompatibles reclaman autoridad sobre el mismo aspecto, el sistema no debería elegir silenciosamente la interpretación más conveniente. Debe detectar el conflicto y resolverlo mediante las reglas correspondientes o mediante una decisión humana.



## I.4. Artefactos y trazabilidad

IASI trabaja con artefactos porque los artefactos permiten conservar el razonamiento de ingeniería fuera de la memoria de una conversación.

No todos los artefactos tienen la misma naturaleza.

Podemos distinguir, al menos, tres grandes clases:

```
fuentes
  inputs

artefactos canónicos de ingeniería
  definitions
  rules
  configuration
  specifications
  tasks
  validations
  states

artefactos derivados
  código
  documentación generada
  configuraciones materializadas
  adapters
  paquetes
  publicaciones
```

La distinción importa especialmente cuando una parte del sistema puede regenerarse.

Si un documento se deriva de unas definitions, modificar manualmente el resultado sin conservar esa decisión rompe la trazabilidad. La siguiente materialización podría perderla.

Por eso IASI intenta conservar una cadena de procedencia:

```
fuente
↓
interpretación
↓
decisión
↓
trabajo
↓
resultado
↓
evidencia
```

La trazabilidad no pretende registrar cada token producido por un modelo.

Pretende poder responder preguntas de ingeniería:

- ¿por qué existe este comportamiento?;
- ¿qué necesidad lo originó?;
- ¿qué specification lo exige?;
- ¿qué task lo materializó?;
- ¿qué validación lo comprobó?;
- ¿qué cambio posterior podría afectarlo?

Cuando estas relaciones pueden reconstruirse, el proyecto deja de depender de recordar «qué hablamos aquel día».

**Parte II.**

**La metodología**



La metodología IASI organiza el paso desde información heterogénea hasta resultados aceptados.

Su secuencia canónica puede representarse así:

```
inputs
  ↓
definitions
  ↓
rules + configuration
  ↓
specifications
  ↓
tasks
  ↓
validations
  ↓
acceptance + states
```

Esta representación no describe una cascada irreversible.

El proceso es iterativo. Una task puede descubrir nueva información; una validation puede revelar que una definition era incompleta; un nuevo input puede afectar specifications ya existentes. IASI conserva esas relaciones para poder volver atrás sin perder la historia.

Los capítulos de esta parte describen cada elemento por su responsabilidad dentro del ciclo.



## II.1. Inputs

Todo proyecto, proceso o actividad de ingeniería comienza con información.

Nada nace de la nada.

Antes de diseñar, decidir o construir existe algún tipo de información que provoca, condiciona o da sentido al trabajo. Puede ser un pliego, una presentación, una idea expresada en unas pocas líneas, una conversación, un correo, una especificación, un diagrama, una hoja de cálculo, código existente, una normativa, una incidencia, una URL o cualquier combinación de ellos.

Esa información no tiene por qué llegar preparada para ser procesada.

Puede aparecer en formatos diferentes, con distintos niveles de detalle y calidad. Puede estar estructurada o ser completamente informal; ser precisa o ambigua; estar completa o contener importantes lagunas; coincidir con otras fuentes o contradecirlas.

IASI denomina **inputs** al conjunto de información que entra en el proceso y que puede ser utilizada para comprender el problema, establecer sus límites y determinar qué acciones pueden realizarse.

**inputs/** constituye, por tanto, **la frontera de entrada del conocimiento en IASI.**

Su objetivo no es normalizar inmediatamente toda esa información ni obligarla a adoptar una forma común. Su función inicial es conservarla, clasificarla por su origen y ponerla a disposición del proceso manteniendo su significado y trazabilidad.

La transformación vendrá después.

```
información heterogénea
↓
inputs/
↓
análisis · contraste · reconciliación
↓
comprensión y normalización
↓
definitions/
```

La separación es fundamental:

**inputs/** conserva la información tal como entra, se aporta o se obtiene.  
**definitions/** representa esa información una vez comprendida y estructurada por IASI.

Los inputs son la materia prima.

Las definitions son conocimiento ya formado para trabajar con él.

### II.1.1. Nombre y ubicación

`inputs` es el nombre utilizado y recomendado por IASI.

Una implementación puede utilizar otro nombre físico cuando exista una razón para hacerlo. Lo importante no es la denominación concreta de la carpeta, sino conservar su semántica y su comportamiento.

La estructura de referencia distingue tres orígenes:

```
inputs/  
  externals/  
  internals/  
  obtained/
```

La clasificación no depende del formato, del contenido ni de la importancia de la información.

Depende exclusivamente de **su procedencia**.

### II.1.2. External inputs

`externals/` contiene la información recibida desde fuera del sistema de trabajo.

Por ejemplo:

- un pliego;
- una petición de un cliente;
- una presentación;
- una especificación recibida;
- documentación de un proveedor;
- una normativa;
- requisitos contractuales;
- un correo;
- documentación de un sistema de terceros;
- cualquier otro material proporcionado como entrada al proyecto.

Un input externo representa **lo que realmente se recibió**.

Por esa razón existe una regla estricta:

**`inputs/externals/` es siempre de solo lectura para IASI.**

IASI puede leer, analizar, relacionar, contrastar e interpretar un input externo.

No puede modificarlo.

No debe:

- corregirlo;
- reescribirlo;
- normalizarlo;
- completarlo;
- eliminar información;
- cambiar su formato para adaptarlo al proceso;
- sustituir silenciosamente una versión por otra.

Si un pliego contiene un error, ese error forma parte del input recibido.

Si una presentación es ambigua, la ambigüedad debe conservarse.

Si dos documentos externos se contradicen, ambos deben permanecer intactos para que la contradicción pueda ser detectada, analizada y resuelta posteriormente.

**Lo externo representa lo que recibimos, no lo que hubiéramos querido recibir.**

Cualquier corrección, interpretación o normalización debe producir **nueva información en otro lugar**.

Nunca debe alterar la evidencia original.

Si posteriormente llega una nueva versión de un documento externo, se incorpora como un nuevo input. La versión anterior se conserva mientras sea necesaria para mantener la trazabilidad del proceso.

### II.1.3. Internal inputs

`internals/` contiene información aportada desde dentro del proceso de ingeniería.

Puede proceder del usuario, del equipo, de una decisión tomada durante el trabajo o de cualquier intervención interna que añada, restrinja o modifique el contexto disponible.

Por ejemplo:

```
Por ahora únicamente crea el proyecto.
```

Un input interno puede:

- añadir información;
- aclarar una petición anterior;
- establecer prioridades;
- introducir restricciones;
- delimitar el alcance;
- cambiar una decisión;
- indicar qué debe hacerse a continuación;
- indicar qué todavía no debe hacerse.

El nuevo input no modifica retroactivamente los anteriores.

Añade nueva información al proceso.

Esto permite conservar cómo evoluciona realmente el conocimiento y las decisiones durante un proyecto.

### II.1.4. Obtained inputs

No toda la información necesaria es proporcionada explícitamente.

Parte de ella se descubre trabajando.

`obtained/` contiene información obtenida mediante actividades como:

- observación;
- análisis;
- inspección;
- exploración;
- ejecución;
- análisis de código;
- análisis de configuración;
- ingeniería inversa;
- pruebas;
- mediciones;
- consultas a sistemas existentes.

Por ejemplo, la documentación recibida puede no indicar qué versión de una biblioteca utiliza un sistema.

Si durante el análisis del repositorio IASI descubre esa versión, esa información no es externa y tampoco ha sido aportada mediante una decisión interna.

Es información **obtenida**.

Esta categoría permite distinguir con claridad:

```
lo que nos dijeron
lo que decidimos
lo que descubrimos
```

Esa diferencia puede ser esencial para la trazabilidad y para evaluar posteriormente la fiabilidad de una conclusión.

### II.1.5. Diferentes formatos, una misma frontera

Los inputs no tienen que adoptar un formato documental único.

Un proyecto puede comenzar con:

```
inputs/
  externals/
    presentation.pdf
    requirements.xlsx
    architecture.png
    contract.pdf
    email.md
  internals/
    001-initial-scope.md
  obtained/
    001-existing-runtime.md
```

IASI debe aceptar que la realidad es heterogénea.

Forzar la normalización antes de incorporar la información produciría un problema importante: dejaríamos de conservar la entrada original y comenzaríamos a confundir **evidencia** con **interpretación**.

Por eso ambos niveles deben permanecer separados.

```
INPUT
"esto es lo que tenemos"
    ↓
ANÁLISIS
"esto es lo que entendemos"
    ↓
DEFINITION
"esto es lo que podemos utilizar"
```

### II.1.6. Información activa e histórica

La información puede dejar de estar vigente sin dejar de ser relevante para la historia del proyecto.

Cada origen puede disponer de un espacio para información archivada:

```
inputs/
  externals/
    archived/
  internals/
    archived/
  obtained/
    archived/
```

Archivar un input no cambia su naturaleza.

Un external archivado continúa siendo external.

Un internal archivado continúa siendo internal.

Un obtained archivado continúa siendo obtained.

Archivar significa únicamente que esa entrada ya no participa en la interpretación del estado vigente.

Los procesos que calculen el contexto activo deben ignorar el contenido archivado.

Su conservación permite reconstruir posteriormente cómo evolucionó la información disponible.

### II.1.7. Los inputs no son la solución

Recibir información no significa disponer de información suficiente para construir una solución.

Por ejemplo:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

Ese input expresa una intención.

Pero puede ser insuficiente para decidir:

- qué gráficos deben generarse;
- qué recibe el sistema como entrada;
- qué produce;
- cuál es la responsabilidad del filtro;
- cuál es la responsabilidad del MCP;
- qué tecnologías deben utilizarse;
- qué restricciones existen;
- cómo debe validarse el resultado.

IASI no debe completar silenciosamente esos huecos mediante suposiciones.

Debe ser capaz de distinguir entre:

```
lo conocido
lo inferido
lo desconocido
```

y determinar qué puede hacerse legítimamente con la información disponible.

### II.1.8. Suficiencia y alcance

La información no necesita estar completa para que el trabajo pueda comenzar.

Necesita ser **suficiente para la acción concreta que se pretende realizar**.

Esta distinción es esencial.

El input:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

puede ser insuficiente para diseñar e implementar el sistema.

Sin embargo, posteriormente puede recibirse:

```
Por ahora únicamente crea el proyecto.
```

Este nuevo input no resuelve las incógnitas de la solución.

Lo que hace es **reducir explícitamente el alcance de la acción**.

IASI sigue sin saber lo suficiente para construir el sistema completo, pero sí puede disponer de información suficiente para crear el proyecto inicial.

Por tanto:

**La falta de información limita el alcance; no necesariamente detiene el proceso.**

Y, de forma más precisa:

**Si no sabemos lo suficiente para realizar una acción, no la realizamos.  
Podemos avanzar únicamente hasta donde la información disponible y los límites establecidos lo permitan.**

Esto permite que IASI avance de forma incremental sin inventar información y sin exigir tampoco una definición completa del proyecto antes de realizar cualquier trabajo.

### II.1.9. Validación antes de actuar

Antes de ejecutar una acción, IASI debe evaluar el conjunto de inputs activos.

Entre otras cuestiones, debe determinar:

- qué información está disponible;
- cuál es su origen;
- qué información sigue vigente;
- si existen contradicciones;
- si existen ambigüedades relevantes;
- qué información falta;
- qué puede inferirse razonablemente;
- qué no puede afirmarse;
- si la información disponible es suficiente para la acción propuesta;
- hasta dónde permite avanzar el alcance establecido.

El objetivo no es conseguir que IASI siempre produzca una acción.

A veces el resultado correcto del análisis será:

```
no existe información suficiente para realizar esta acción
```

En otras ocasiones podrá concluir:

```
no puedo implementar la solución,  
pero sí puedo realizar esta parte concreta
```

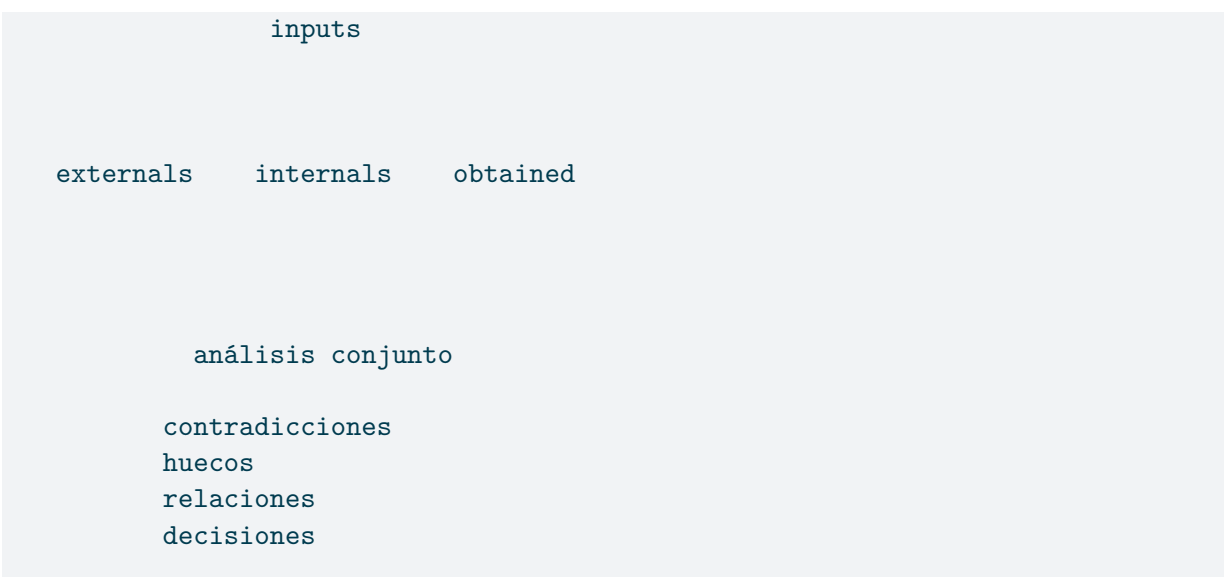
Ambos resultados representan comportamiento correcto.

### II.1.10. De inputs a definitions

Los inputs no deben ser utilizados como sustituto permanente de una representación estructurada del proyecto.

IASI debe analizarlos, relacionarlos, contrastarlos y reconciliarlos.

De ese proceso nacen las **definitions**.



## definitions

La relación no es necesariamente uno a uno.

Un único input puede contribuir a varias definitions.

Una definition puede construirse a partir de numerosos inputs.

Un input puede incluso no producir ninguna definition si resulta irrelevante, está superado o únicamente sirve como evidencia histórica.

IASI debe conservar siempre la diferencia entre ambos niveles:

**Inputs responden a: “¿de dónde sale lo que sabemos?”**

**Definitions responden a: “¿qué hemos entendido y estructurado para poder trabajar?”**

Mantener esa frontera permite que el conocimiento evolucione sin destruir sus fuentes y proporciona una base sólida para la trazabilidad, la validación y la reproducibilidad del proceso de ingeniería.

## II.2. Definitions

Los inputs conservan información.

Las definitions representan **lo que IASI ha comprendido y estructurado a partir de esa información.**

La relación no es uno a uno.

Un único input puede contribuir a varias definitions y una definition puede necesitar múltiples inputs, decisiones y observaciones para quedar suficientemente formada.

```
inputs
  external
  internal
  obtained
    ↓
  análisis · contraste · reconciliación
    ↓
definitions
```

Una definition no debería ser una copia cosmetizada del documento de origen.

Su función es hacer explícito conocimiento utilizable por el resto del proceso: conceptos, restricciones, relaciones, límites, decisiones consolidadas, incógnitas relevantes o cualquier otra estructura que permita razonar con mayor precisión.

### II.2.1. Conocido, inferido y desconocido

IASI debe conservar la diferencia entre:

```
conocido
inferido
desconocido
```

Una inferencia puede ser útil, pero no debe adquirir silenciosamente la misma autoridad que una evidencia o una decisión confirmada.

Del mismo modo, una incógnita relevante debe poder permanecer explícita. La metodología no exige rellenar todos los huecos antes de comenzar; exige no ocultarlos.

### II.2.2. Generación y revisión

Las definitions pueden materializarse mediante un skill, un workflow, una herramienta o trabajo humano.

El mecanismo utilizado no cambia su responsabilidad.

El Sistema Inteligente puede proponer una interpretación. La condición canónica procede del proceso que la revisa, acepta y relaciona con sus fuentes, no de la confianza aparente con la que fue redactada.

### II.2.3. Evolución

Cuando cambia un input activo puede ser necesario revisar las definitions que dependen de él. Ese cambio puede propagarse hacia specifications, tasks, validaciones o artefactos derivados. IASI no necesita regenerarlo todo indiscriminadamente. Necesita saber **qué queda afectado**.

## II.3. Rules y configuración

Comprender el problema no basta para actuar.

El trabajo se realiza dentro de normas, convenciones, restricciones y parámetros que también deben hacerse explícitos.

IASI separa dos conceptos relacionados.

### II.3.1. Rules

Una **rule** expresa una norma que debe, no debe o conviene respetar dentro de un ámbito.

Puede regular estilo, arquitectura, seguridad, documentación, compatibilidad, procesos o cualquier otro aspecto repetible de la ingeniería.

Las reglas pueden ser generales o específicas. Cuando dos reglas regulan el mismo aspecto, **la más específica puede prevalecer sobre la genérica**.

Por ejemplo, una regla de estilo específica para Python puede refinar una regla de estilo genérica.

La plantilla normalizada de una rule puede contener:

```
título
metadata
rule
examples
exceptions
rationale
sources
```

Los ejemplos son opcionales, pero resultan útiles para mostrar casos positivos, negativos o límites de aplicación.

### II.3.2. Configuration

La configuración expresa valores concretos que parametrizan el trabajo.

Una rule puede establecer que una tecnología crítica debe permanecer fijada durante un proyecto. La configuración indica **qué versión concreta** está fijada.

Una rule expresa política.

La configuration materializa parámetros.

Separar ambas permite cambiar valores sin reescribir la norma y reutilizar la misma norma en proyectos distintos.



## II.4. Specifications

Una **specification** expresa **qué debe quedar satisfecho** por una unidad de trabajo o por un resultado.

No es una lista de ocurrencias del agente ni un registro de cómo decidió implementar algo.

Parte de:

```
definitions
+ rules
+ configuration
+ alcance
    ↓
specification
```

La specification transforma conocimiento y restricciones en una expectativa verificable.

Debe permitir responder, al menos, a estas preguntas:

- ¿qué resultado se espera?;
- ¿qué alcance tiene?;
- ¿qué restricciones son aplicables?;
- ¿qué queda explícitamente fuera?;
- ¿cómo sabremos que se ha satisfecho?

### II.4.1. Specification y solución

Es importante no confundir una specification con la implementación.

La specification puede limitar tecnologías o imponer decisiones cuando existe una razón para ello, pero no debería describir innecesariamente cada paso de ejecución.

Su responsabilidad es establecer el contrato del resultado.

### II.4.2. Criterios de aceptación

Siempre que sea posible, una specification debe conducir a criterios que permitan validar el resultado.

Una frase como «mejorar el rendimiento» es débil si no sabemos qué comportamiento debe cambiar ni cómo evaluar que la mejora es suficiente.

La metodología intenta que el salto entre intención y aceptación pueda recorrerse sin depender de interpretaciones retrospectivas.

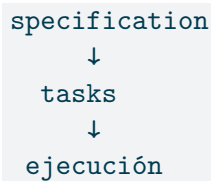


## II.5. Tasks

Las **tasks** convierten una **specification** en unidades de trabajo ejecutables.

Una **task** no crea por sí sola nuevo alcance.

Debe poder relacionarse con una necesidad, una **specification** o una decisión que justifique su existencia.



El tamaño adecuado de una **task** no se define por número de archivos o líneas de código.

Debe ser suficientemente concreta para poder:

- saber cuándo comienza;
- conocer qué pretende producir;
- identificar sus dependencias;
- ejecutarla de forma controlada;
- validar su resultado;
- determinar cuándo puede cerrarse.

### II.5.1. Ejecución y estado

Una **task** atraviesa estados.

La capacidad para ejecutar una **task** no concede automáticamente permiso para cerrarla. La transición de estado debe respetar las validaciones y las reglas del **workflow** correspondiente.

IASI puede permitir que distintas personas, agentes o herramientas ejecuten **tasks** diferentes.

La unidad metodológica sigue siendo la **task** y su contrato, no la identidad de quien materializa el trabajo.

### II.5.2. Resultado

La ejecución produce un **outcome**.

El **outcome** es el resultado observado de la operación. No debe confundirse con un argumento inventado para justificar retrospectivamente que una **task** está terminada.

Esta distinción será importante al estudiar los **commands** de **task**.



## II.6. Validations

Una **validation** aporta evidencia sobre una condición que debe cumplirse.

IASI no considera que un artefacto sea correcto porque haya sido generado correctamente desde el punto de vista sintáctico ni porque un agente afirme haber terminado.

El resultado debe poder enfrentarse a criterios explícitos.

Las validations pueden comprobar, entre otras cosas:

- estructura;
- sintaxis;
- compilación;
- pruebas;
- reglas;
- consistencia entre artefactos;
- criterios de aceptación;
- presencia o ausencia de determinados efectos.

### II.6.1. Validar no es generar

Separar generación y validación reduce un riesgo importante: que el mismo proceso que produjo una decisión se limite a declararla correcta.

No siempre será posible utilizar mecanismos completamente independientes, pero la metodología mantiene conceptualmente ambas responsabilidades.

```
ejecución
  ↓
resultado
  ↓
validation
  ↓
evidencia
```

### II.6.2. Fallar también es un resultado

Una validation fallida no es un accidente que deba ocultarse.

Es información del proceso.

Puede bloquear transiciones posteriores, devolver trabajo a una fase anterior o exigir nueva información.

IASI debe conservar suficiente estado para que el fallo no desaparezca simplemente porque se inicia una nueva conversación o cambia el agente.



## II.7. Estados y aceptación

Los estados permiten saber **dónde se encuentra el trabajo** sin reconstruir toda su historia cada vez.

Pero un estado no debe convertirse en una etiqueta decorativa.

Para que sea significativo debe estar respaldado por las condiciones que permiten alcanzarlo.

Una task puede estar iniciada, en ejecución, completada, fallida, reabierta o en cualquier otro estado definido por su ciclo de vida. Lo importante no es el vocabulario concreto, sino que las transiciones sean explícitas y reproducibles.

### II.7.1. Done no significa necesariamente accepted

Completar el trabajo y aceptarlo son responsabilidades diferentes.

**done** puede expresar que la ejecución prevista ha finalizado.

La aceptación expresa que el resultado satisface las condiciones que justifican su incorporación al estado válido del proyecto.

```
trabajo realizado
  ↓
validaciones
  ↓
evidencia
  ↓
aceptación
  ↓
estado consolidado
```

Esta separación permite que IASI conserve una frontera clara entre **hacer** y **dar por bueno**.

### II.7.2. El estado es materializado

El estado relevante no debe depender de que una persona o un agente recuerden qué ocurrió.

Debe quedar persistido de forma que el siguiente participante pueda continuar el trabajo con el mismo punto de partida observable.



## II.8. Cambio e iteración

Los proyectos cambian.

Llegan nuevos documentos, se toman decisiones, aparecen restricciones, se descubren datos y se invalidan supuestos anteriores.

IASI no intenta eliminar ese cambio. Intenta **hacer visible su impacto**.

Cuando cambia una fuente, la pregunta no es solamente «¿qué fichero debo regenerar?».

La pregunta es:

**¿qué conocimiento, decisiones y resultados dependen de aquello que ha cambiado?**

```
nuevo input
  ↓
definitions afectadas
  ↓
specifications afectadas
  ↓
tasks / validations afectadas
  ↓
artefactos derivados
```

### II.8.1. Invalidación

Una validación puede ser correcta para un conjunto concreto de entradas y dejar de ser válida cuando esas entradas cambian.

Por eso determinados estados pueden asociarse a hashes o identificadores de las fuentes que justificaron su resultado.

Cambiar la dependencia invalida la conclusión anterior.

### II.8.2. Archivar no es borrar

Un input que deja de estar activo puede archivar sin perder su naturaleza ni su historia.

El material archivado deja de participar en el contexto vigente, pero sigue permitiendo reconstruir cómo evolucionó el proyecto.

### II.8.3. Contenido parcialmente derivable

Algunos artefactos combinan partes derivables y contenido editorial o profesional añadido manualmente.

IASI debe regenerar únicamente aquello que pueda derivarse de forma segura.

Si cambia una dependencia de contenido mantenido por autor, el comportamiento correcto puede ser marcarlo como **affected** y solicitar revisión, no reescribirlo silenciosamente.



**Parte III.**

## **Estructura de un proyecto IASI**



La metodología define responsabilidades.

La estructura del proyecto les proporciona un lugar estable.

IASI utiliza directorios y artefactos con significado conocido para que personas, Sistemas Inteligentes y herramientas puedan encontrar información sin depender de convenciones implícitas de cada equipo.

La estructura es un **contrato operativo**, no una decoración del repositorio.

No todos los proyectos necesitan materializar todos los directorios desde el primer minuto. La estructura puede crecer conforme aparecen capacidades y artefactos. Lo importante es que, cuando una categoría existe, conserve su semántica.



## III.1. La raíz del proyecto

Una estructura de referencia puede tomar esta forma:

```
project/  
  .iasi/  
  inputs/  
  definitions/  
  rules/  
  configuration/  
  specifications/  
  tasks/  
  validations/  
  states/  
  outputs/  
  releases/  
  .codex/  
  .copilot/
```

La presencia concreta depende de las capacidades utilizadas.

IASI no exige crear carpetas vacías para simular completitud.

### III.1.1. Dos familias de directorios

Conviene distinguir:

#### Artefactos de ingeniería

```
inputs/  
definitions/  
rules/  
configuration/  
specifications/  
tasks/  
validations/  
states/  
outputs/  
releases/
```

representan información y resultados del proyecto.

#### Infraestructura de IASI y de los entornos

```
.iasi/  
.codex/  
.copilot/  
...
```

permite que la metodología se materialice y sea utilizada por herramientas concretas.

La diferencia es importante porque las carpetas específicas de un agente no deben convertirse en la fuente de verdad del proyecto.

### III.1.2. Estructura semántica

Cambiar un nombre físico puede ser posible cuando exista una razón y la configuración lo soporte.

Pero hacerlo no debe borrar la semántica.

Un directorio llamado de otra manera sigue necesitando una responsabilidad inequívoca. IASI no debería tener que adivinar si una carpeta contiene evidencia original, conocimiento normalizado o resultados derivados.

### III.1.3. Estructura mínima y estructura creciente

La estructura de referencia no significa que todo proyecto deba comenzar con todas las carpetas vacías.

IASI distingue entre **semántica requerida** y **materialización necesaria en cada momento**.

Un proyecto crea los espacios que necesita conforme utiliza las capacidades correspondientes. Lo importante es que, cuando exista un artefacto, su ubicación y su significado sean inequívocos.

Por tanto, la estructura puede crecer con el proyecto sin perder el contrato común.

## III.2. Estructura de inputs

La estructura de referencia de los inputs distingue su procedencia:

```
inputs/  
  externals/  
    archived/  
  internals/  
    archived/  
  obtained/  
    archived/
```

La clasificación responde a tres preguntas diferentes:

```
externals → ¿qué recibimos?  
internals → ¿qué aportamos o decidimos?  
obtained  → ¿qué descubrimos trabajando?
```

Los subárboles `archived/` conservan información histórica que ya no participa en el contexto activo.

El archivado no cambia la naturaleza del input.

Un external archivado continúa siendo external. Simplemente deja de intervenir en el cálculo vigente.

### III.2.1. Solo lectura de externals

Desde el punto de vista de IASI:

**inputs/externals/ es de solo lectura.**

Cualquier corrección, normalización o interpretación produce un nuevo artefacto fuera de ese directorio.

Esta propiedad permite utilizar los inputs externos como evidencia estable incluso cuando el conocimiento derivado evoluciona.



### III.3. Artefactos de ingeniería

Después de la frontera de inputs aparecen los directorios que materializan el proceso de ingeniería:

```
definitions/
rules/
configuration/
specifications/
tasks/
validations/
states/
```

No forman una colección arbitraria.

Cada uno responde a una pregunta distinta:

| Directorio      | Pregunta                               |
|-----------------|----------------------------------------|
| definitions/    | ¿qué hemos comprendido?                |
| rules/          | ¿qué normas debemos respetar?          |
| configuration/  | ¿con qué parámetros trabajamos?        |
| specifications/ | ¿qué debe quedar satisfecho?           |
| tasks/          | ¿qué trabajo concreto ejecutamos?      |
| validations/    | ¿qué evidencia comprueba el resultado? |
| states/         | ¿cuál es el estado materializado?      |

La metodología puede añadir especialización interna dentro de cada uno.

Por ejemplo, las rules pueden organizarse por dominio, tecnología o lenguaje. Esa organización no cambia su naturaleza normativa.

#### III.3.1. No confundir estructura con secuencia

Que estos directorios puedan representarse en un flujo no significa que únicamente puedan escribirse en ese orden.

Una validación puede descubrir información que produzca un nuevo input obtenido. Una modificación de configuration puede afectar specifications existentes. Un cambio de rule puede invalidar trabajo previamente aceptado.

La estructura conserva responsabilidades.

El workflow decide cómo se recorren.



## III.4. Estado local de IASI

`.iasi/` contiene información de control necesaria para que IASI opere dentro de un proyecto.

Parte de esa información puede componerse o heredarse entre ámbitos. Otra debe ser estrictamente local.

Un ejemplo importante es:

```
.iasi/validation.json
```

El resultado persistente de validación pertenece al proyecto o ámbito que fue validado.

**No se hereda.**

Heredar un estado de validación produciría una conclusión falsa: que un hijo está validado porque lo estaba su padre aunque sus entradas, reglas o contenido sean distintos.

### III.4.1. Composición entre ámbitos

Cuando existen configuraciones IASI en distintos niveles, pueden componerse de padre a hijo.

La resolución se realiza desde el ámbito más general hacia el más próximo, y el más próximo puede refinar aquello que resulte heredable.

Este mecanismo permite compartir comportamiento sin duplicarlo en todos los proyectos.

Pero la herencia no debe aplicarse indiscriminadamente.

Cada artefacto debe declarar o tener definida su semántica de composición.



## III.5. Directorios derivados de adapters

Un proyecto puede utilizar uno o varios agentes o entornos.

Por ejemplo:

```
.iasi/  
.codex/  
.copilot/
```

La relación entre ellos no es simétrica.

**.iasi/ representa la información canónica de IASI; los directorios específicos de herramientas son proyecciones derivadas.**

Un adapter puede materializar prompts, instrucciones, configuraciones, skills u otros archivos que un entorno concreto necesite.

Esos archivos existen para integrar IASI con la herramienta.

No deben convertirse en el lugar donde se redefine la metodología.

```
IASI canónico  
  ↓  
  adapter  
  ↓  
.codex/ · .copilot/ · ...
```

Si cambia el formato requerido por una herramienta, cambia el adapter.

No debería ser necesario reescribir el command o workflow canónico para seguir utilizándolo.



## III.6. Outputs y releases

`outputs/` y `releases/` representan momentos diferentes del ciclo de vida de un resultado.

### III.6.1. Outputs

`outputs/` contiene resultados materializados por el trabajo.

Pueden ser código, documentos, paquetes, informes, binarios u otros artefactos derivados.

Que algo aparezca en `outputs/` no significa necesariamente que esté aceptado o publicado.

### III.6.2. Releases

`releases/` contiene resultados preparados para entrega o publicación según las reglas del proyecto.

```
trabajo
  ↓
outputs/
  ↓
validación y aceptación
  ↓
releases/
```

### III.6.3. Evitar realimentación accidental

Las herramientas utilizadas para construir un resultado pueden producir sus propios directorios temporales.

Esos artefactos intermedios no deberían mezclarse con la salida final si eso hace que vuelvan a entrar como fuente del siguiente build.

Cuando una herramienta necesita un directorio de construcción, conviene aislarlo y enrutar únicamente el resultado final hacia `outputs/`.

La regla general es sencilla:

**Los resultados derivados no deben reaparecer accidentalmente como nuevas entradas del mismo proceso de materialización.**



**Parte IV.**

**Workflows**



La metodología define las responsabilidades y los artefactos.

Un workflow define **cómo recorreremos esas responsabilidades para alcanzar un objetivo concreto**.

Dos workflows pueden utilizar los mismos inputs, rules, commands y validations en órdenes o combinaciones diferentes sin que la metodología cambie.

```
metodología
  ↓
workflows
  ↓
ejecuciones concretas
```

Un workflow puede incluir trabajo humano, llamadas a un Sistema Inteligente, herramientas deterministas, validaciones automáticas y decisiones que requieran autoridad profesional.



## IV.1. Modelo de workflow

Un workflow es una secuencia coordinada de actividades con condiciones de entrada, transiciones y criterios de salida.

No es simplemente una lista de pasos escrita en un prompt.

Un workflow debería poder hacer explícitos:

- objetivo;
- entradas requeridas;
- artefactos que puede leer;
- artefactos que puede modificar;
- commands que utiliza;
- validaciones necesarias;
- estados que puede producir;
- puntos de decisión;
- condiciones de finalización.

La misma definición puede ejecutarse de formas diferentes.

Una persona puede realizar manualmente determinados pasos. Un agente puede ejecutar otros. Una automatización puede encadenarlos cuando existan condiciones suficientes.

La semántica del workflow no depende de quién pulsa el botón.

### IV.1.1. Workflow y metodología

La metodología define las reglas del juego.

El workflow define una forma válida de jugar una partida.

Por eso un workflow no puede saltarse una validación obligatoria simplemente porque el agente utilizado tenga capacidad técnica para continuar.



## IV.2. De inputs a definitions

Uno de los workflows fundamentales transforma información de entrada en conocimiento utilizable.

```
inputs activos
  ↓
validar disponibilidad y origen
  ↓
detectar contradicciones y huecos
  ↓
analizar y relacionar
  ↓
proponer definitions
  ↓
revisar / aceptar
```

El objetivo no es resumir todos los documentos.

Es producir definitions que permitan continuar el trabajo sin perder la relación con sus fuentes.

El workflow debe poder terminar también sin crear una definition.

Eso puede ocurrir cuando el input es irrelevante, está superado, contiene información insuficiente o únicamente aporta evidencia histórica.

La ausencia de un artefacto nuevo puede ser un resultado correcto.



## IV.3. Planificación y ejecución

Una vez existe conocimiento suficiente, IASI puede convertir una necesidad en trabajo ejecutable.

```
definitions
+ rules
+ configuration
  ↓
specifications
  ↓
tasks
  ↓
ejecución
```

La planificación no necesita descomponer por adelantado todo el proyecto hasta el último detalle.

Puede materializar únicamente aquello que el alcance actual permite decidir con suficiente confianza.

Las tasks se ejecutan dentro del contrato de su specification y respetando las rules aplicables.

El ejecutor puede ser una persona, un Sistema Inteligente, una herramienta o una combinación de ellos.

Lo importante es que el resultado vuelva al proceso como un artefacto observable y pueda ser validado.



## IV.4. Validación y aceptación

El workflow de validación comienza cuando existe un resultado que debe contrastarse con una condición.

```
resultado
  ↓
validations aplicables
  ↓
evidencia
  fail → revisar / corregir
  pass → candidato a aceptación
        ↓
        aceptar
        ↓
        state
```

El paso de **pass** a aceptación no debe confundirse con una operación puramente mecánica.

Hay contextos en los que la aceptación puede automatizarse porque los criterios son completamente objetivos y la autoridad ha sido delegada de forma explícita.

En otros, requiere decisión humana.

IASI debe permitir ambos casos sin ocultar quién o qué tenía autoridad para efectuar la transición.



## IV.5. Workflow de cambio

Un cambio comienza con nueva información, no con la obligación de regenerar todo el proyecto.

```
nuevo input
  ↓
clasificar origen
  ↓
incorporar sin destruir historia
  ↓
analizar impacto
  ↓
marcar affected / invalidar
  ↓
actualizar únicamente lo necesario
  ↓
volver a validar
```

La propagación depende de las relaciones de trazabilidad.

Si una definition no depende del input modificado, no existe razón metodológica para reconstruirla.

Si un artefacto es parcialmente derivable, puede regenerarse su parte automática y conservar la parte de autor.

El workflow de cambio convierte la trazabilidad en una herramienta operativa: permite avanzar sin tratar cada modificación como si el proyecto comenzara de cero.



**Parte V.**

**Commands**



Los commands proporcionan un vocabulario canónico para solicitar operaciones IASI.

Un command no pertenece a Codex, Copilot, Claude ni a ningún otro agente.

Su significado pertenece a IASI.

El adapter decide cómo presentar o invocar esa operación en cada entorno.

```
command IASI
  ↓
  adapter
  ↓
sintaxis / mecanismo del entorno
```

Esta separación permite que el mismo workflow sobreviva al cambio de herramienta.



## V.1. Modelo de command

Un command debe poder describirse sin depender de la interfaz que lo invoque.

Conceptualmente contiene:

```
nombre  
intención  
parámetro principal  
parámetros adicionales  
precondiciones  
efectos  
resultado  
transiciones permitidas
```

### V.1.1. Parámetro principal

Cuando un command posee un parámetro natural, puede declararlo como parámetro principal.

Esto permite utilizar binding posicional o nombrado sin introducir ambigüedad.

### V.1.2. Literales y referencias

IASI distingue conceptualmente entre un literal y una referencia.

Una cadena entre comillas representa un valor literal.

Un identificador sin comillas puede representar una referencia resoluble por el command.

### V.1.3. Selectors

Los selectors permiten aplicar una operación a cero, uno o varios artefactos.

Un selector que no encuentra coincidencias no debe transformarse silenciosamente en otra operación. El resultado puede ser un warning y ninguna modificación.

### V.1.4. Outcome

El outcome describe lo que ocurrió al ejecutar la operación.

No es un argumento que el llamador pueda inventar para imponer el estado final.



## V.2. validate

/validate fue el primer command canónico definido para IASI.

Su responsabilidad es comprobar que las condiciones necesarias permiten continuar el proceso.

En su forma inicial evalúa:

```
instructions
+
inputs activos
```

y materializa un resultado persistente:

```
.iasi/validation.json
```

### V.2.1. Fallo persistente

Si la validación falla, el estado no desaparece al terminar la conversación.

El resultado persistente bloquea las fases posteriores que dependan de esa validación.

Esto evita que un nuevo agente o una nueva sesión continúen simplemente porque no conocen el fallo anterior.

### V.2.2. Invalidación por cambio

Una validación solo es válida respecto al conjunto de entradas que fue comprobado.

IASI puede utilizar hashes de esas dependencias.

Si cambian, la validación anterior deja de ser aplicable y debe ejecutarse de nuevo.

### V.2.3. Estado local

.iasi/validation.json es local.

No debe heredarse desde un proyecto padre, porque la validez de un ámbito no demuestra la validez de otro.



## V.3. Commands de task

IASI dispone de un pequeño vocabulario para controlar el ciclo de vida de una task.

La forma canónica acordada es:

```
task.start(selector)
task.run(selector, completion = "auto")
task.done(selector)
task.rerun(selector, completion = "auto")
task.reopen(selector)
```

### V.3.1. `task.start`

Inicia una task seleccionada cuando sus precondiciones permiten comenzar.

### V.3.2. `task.run`

Ejecuta la task.

`completion = "auto"` permite que la operación complete automáticamente la transición prevista cuando las condiciones del workflow lo permitan.

### V.3.3. `task.done`

Materializa la finalización de una task.

El outcome procede de la ejecución y de la evidencia observada. No se introduce como argumento de `task.done` para forzar una conclusión.

### V.3.4. `task.rerun`

Permite volver a ejecutar trabajo desde estados como `done` o `failed` cuando el workflow lo autoriza.

### V.3.5. `task.reopen`

Reabre trabajo que ya había sido cerrado para permitir una nueva intervención explícita.

La existencia de commands diferentes hace visibles transiciones que de otro modo quedarían ocultas dentro del comportamiento del agente.



## V.4. archive

Archivar un input significa retirarlo del contexto activo sin destruir su historia.

La operación mantiene la rama lógica del documento:

```
inputs/externals/... → inputs/externals/archived/...  
inputs/internals/... → inputs/internals/archived/...  
inputs/obtained/... → inputs/obtained/archived/...
```

El nombre archivado puede incorporar una marca temporal `YYYYMMDDhhmmss` para conservar versiones sin colisiones.

### V.4.1. Efecto sobre la validación

Los subárboles `archived/` se excluyen del conjunto de inputs activos y, por tanto, de los hashes utilizados para validar el estado vigente.

Archivar no reescribe el documento.

No cambia su procedencia.

Solo modifica su participación en el contexto activo.



## V.5. adapter

El command canónico para materializar la integración con un agente adopta la forma:

```
adapter(agent, dir=".<agent>", overwrite=false)
```

Por ejemplo:

```
adapter(agent="codex", dir=".codex", overwrite=false)
```

El command selecciona un adapter conocido y proyecta sus artefactos sobre el directorio de destino.

### V.5.1. overwrite

Por defecto, la materialización no debe destruir silenciosamente contenido existente.

`overwrite=false` obliga a tratar una colisión como una condición explícita.

### V.5.2. Responsabilidad

`adapter` no cambia la metodología.

Materializa una representación compatible con un entorno concreto a partir de información canónica de IASI.

Por eso el resultado del adapter es **derivado** y puede regenerarse.



**Parte VI.**

## **Adapters e integración**



IASI pretende vivir sobre cualquier agente y, cuando sea necesario, sobre distintos modelos o runtimes.

Eso solo es posible si la metodología no adopta como propios los formatos particulares de una herramienta.

Los adapters forman esa frontera.

**IASI no se adapta metodológicamente al agente. El adapter adapta el agente a IASI.**

Esta parte desarrolla cómo se conserva una semántica canónica mientras cambian los entornos de ejecución.



## VI.1. Por qué existen los adapters

Cada agente posee convenciones diferentes.

Puede buscar instrucciones en directorios distintos, utilizar formatos propios para commands o skills, ofrecer herramientas diferentes y tener mecanismos específicos para cargar contexto.

Si la metodología dependiera directamente de esas convenciones, terminaríamos manteniendo varias IASI:

```
IASI para Codex  
IASI para Copilot  
IASI para Claude  
IASI para ...
```

El adapter evita esa fragmentación.

```
semántica IASI  
  ↓  
  adapter  
  ↓  
semántica del host
```

El núcleo define la operación.

El adapter conoce el idioma operativo del entorno.



## VI.2. Canónico y derivado

La distinción principal es:

```
.iasi/      → canónico dentro del proyecto  
.codex/     → proyección derivada  
.copilot/   → proyección derivada  
...
```

Los nombres concretos de los directorios derivados pueden configurarse, pero su naturaleza no cambia.

Un archivo generado para un agente no debe adquirir autoridad simplemente porque ese agente lo consume directamente.

Si existe una discrepancia entre la representación derivada y la fuente canónica, debe corregirse la fuente o el adapter y volver a materializarse.

Modificar únicamente la proyección crea una bifurcación invisible.

### VI.2.1. Regenerabilidad

Una buena proyección debe poder reconstruirse.

Eso convierte los directorios de adapter en resultados controlados, no en nuevos centros de conocimiento.



## VI.3. Adapters de agente

Un adapter de agente traduce capacidades IASI a los mecanismos que un entorno concreto ofrece.

Puede materializar:

- instrucciones;
- commands;
- skills;
- configuración;
- referencias a tools;
- convenciones de directorio;
- otros artefactos requeridos por el host.

La traducción debe ser lo más delgada posible.

El adapter no debería contener una segunda implementación independiente de la metodología.

Su trabajo consiste en proyectar la definición canónica y resolver diferencias de formato o integración.

### VI.3.1. Adaptación atómica

La materialización de un adapter debe tratarse como una operación controlada.

Si no puede producir una proyección coherente, es preferible fallar de forma explícita que dejar un entorno a medio adaptar y permitir que el agente trabaje con una mezcla de versiones.



## VI.4. Agente, modelo y runtime

Agente y modelo no son la misma dependencia.

Un agente puede utilizar diferentes modelos.

Un mismo modelo puede aparecer detrás de agentes o runtimes distintos.

IASI evita acoplar ambas decisiones.

```
IASI
↓
adapter de agente / entorno
↓
runtime
↓
modelo o proveedor
```

En algunos casos el adapter necesita conocer capacidades del runtime o del modelo disponible.

Eso no convierte esas capacidades en requisitos universales de la metodología.

La separación permite sustituir un proveedor de modelo sin tener que redefinir el significado de una task, una validation o un command.



## VI.5. Capacidades y degradación

No todos los entornos pueden hacer lo mismo.

Un agente puede disponer de edición de archivos, ejecución de shell, acceso a red, MCP, memoria persistente o ninguna de esas capacidades.

El adapter debe conocer las capacidades relevantes antes de prometer una operación.

### VI.5.1. No simular garantías

Cuando falta una capacidad necesaria, IASI no debería fingir que la garantía se mantiene.

Las respuestas posibles dependen del workflow:

- utilizar un mecanismo alternativo equivalente;
- degradar explícitamente la operación;
- solicitar intervención humana;
- bloquear la transición.

Lo importante es que la diferencia sea visible.

**La portabilidad no consiste en fingir que todos los agentes son iguales.  
Consiste en conservar la semántica de IASI a pesar de que no lo sean.**



**Parte VII.**

**Mecanismos transversales**



Algunos elementos de IASI no pertenecen a una única fase del ciclo.

Rules, skills, templates, configuration y automations pueden intervenir en numerosos workflows y afectar a múltiples clases de artefactos.

Esta parte los estudia como mecanismos reutilizables.

También recoge dos problemas transversales: cómo resolver alcance y precedencia y cómo combinar contenido derivado con aportaciones mantenidas por personas.



## VII.1. Rules

Una rule materializa una norma reutilizable.

Su estructura normalizada puede expresarse en Markdown mediante:

```
# <Rule name>

## Metadata
## Rule
## Examples
## Exceptions
## Rationale
## Sources
```

**Examples** es opcional. Puede contener casos positivos, negativos o situaciones límite cuando ayuden a comprender la norma.

### VII.1.1. Nivel normativo

La metadata puede distinguir niveles como:

```
must
must-not
should
should-not
may
```

Esto evita expresar con el mismo peso una obligación y una recomendación.

### VII.1.2. Especificidad

Cuando una rule específica y otra genérica regulan el mismo aspecto, IASI puede aplicar la más específica.

Por ejemplo:

```
coding/styles/generic/*
coding/styles/python/*
```

Para código Python, la regla específica del lenguaje prevalece cuando ambas entran en conflicto sobre el mismo asunto.

La especificidad permite reutilizar principios generales sin impedir que un dominio introduzca sus propias restricciones.



## VII.2. Skills

Un skill encapsula conocimiento operativo reutilizable para realizar una clase de trabajo.

No es una fuente de verdad del proyecto.

Es un mecanismo que ayuda a transformar artefactos respetando la metodología.

Por ejemplo, la generación de definiciones a partir de inputs puede apoyarse en un skill que indique cómo:

- identificar fuentes;
- extraer conceptos;
- detectar contradicciones;
- conservar incertidumbre;
- relacionar cada conclusión con evidencia;
- producir la estructura canónica esperada.

El skill puede ser utilizado por distintos agentes.

Su valor reside precisamente en que la forma de trabajar no queda encerrada en el prompt improvisado de una sesión.

### VII.2.1. Skill y command

Un command expresa una operación.

Un skill aporta conocimiento para ejecutarla.

Pueden colaborar, pero no son equivalentes.



## VII.3. Templates

Los templates proporcionan formas canónicas para artefactos repetibles.

Una plantilla no debería obligar a rellenar contenido inexistente.

Su función es estabilizar estructura, metadata y convenciones allí donde esa estabilidad aporta valor.

Por ejemplo, una plantilla de rule puede garantizar que todas las rules expresen de la misma forma su ámbito, nivel normativo, excepciones y fuentes.

### VII.3.1. Plantilla y semántica

La plantilla determina la forma esperada.

No decide el contenido.

Un Sistema Inteligente puede utilizarla para materializar un nuevo artefacto, pero sigue necesitando inputs y autoridad suficientes para completar cada sección.

### VII.3.2. Templates incorporados

IASI puede distribuir templates de fábrica dentro de su contenido `builtin/`.

Un proyecto puede reutilizarlos o especializarlos sin convertir una copia modificada en una excepción oculta del runtime.



## VII.4. Configuration

La configuration reúne parámetros que modifican el comportamiento del proceso sin redefinir su metodología.

Puede fijar:

- versiones;
- rutas;
- nombres físicos;
- opciones de herramientas;
- valores por defecto;
- capacidades habilitadas;
- otras decisiones parametrizables.

Una configuración importante debe ser persistente y revisable.

### VII.4.1. Estabilidad tecnológica

Durante un proyecto, las tecnologías críticas deberían mantenerse fijadas salvo que exista una necesidad justificada para cambiarlas.

Que aparezca una versión más reciente no convierte automáticamente la actualización en una buena decisión.

Un cambio de versión puede afectar compatibilidad, arquitectura, dependencias, pruebas, despliegue o certificaciones.

La configuration permite que esa decisión sea explícita en lugar de depender del entorno accidental de quien ejecuta el trabajo.



## VII.5. Automations

Una automation ejecuta de forma automática una operación o parte de un workflow cuando se cumplen determinadas condiciones.

No debe confundirse con el workflow.

El workflow define la lógica del recorrido.

La automation elimina intervención manual en uno o varios puntos de ese recorrido.

```
workflow
  actividad humana
  command
  automation
  validation
```

Automatizar no cambia la autoridad de una transición.

Si una aceptación requiere decisión humana, envolver el proceso en una automation no elimina ese requisito.

La automatización es válida cuando conserva las precondiciones, validaciones y límites establecidos por la metodología.



## VII.6. Ámbito y precedencia

IASI necesita combinar información procedente de distintos niveles.

Puede existir configuración común para varios proyectos, reglas compartidas y especializaciones locales.

La resolución debe ser explícita.

### VII.6.1. De lo general a lo próximo

En estructuras componibles, IASI puede construir el contexto desde el ámbito padre hacia el hijo.

El ámbito más próximo puede refinar aquello que sea heredable.

```
general
  ↓
organización
  ↓
proyecto
  ↓
subproyecto
```

No todo se hereda.

El estado local de validación es un ejemplo claro de información que debe permanecer vinculada al ámbito concreto.

### VII.6.2. Especificidad

La proximidad no es la única dimensión.

Las reglas pueden resolver también por especificidad semántica: una regla de lenguaje es más específica que una regla genérica de coding para ese lenguaje.

### VII.6.3. Resolver, no adivinar

Cuando no existe una regla de precedencia aplicable y dos fuentes incompatibles reclaman autoridad sobre el mismo aspecto, el resultado correcto es un conflicto explícito.

IASI no debería esconderlo escogiendo una opción al azar.



## VII.7. Contenido derivado y contenido de autor

No todos los artefactos son completamente generables ni completamente manuales.

Una guía de usuario, por ejemplo, puede derivar gran parte de su estructura y contenido del conocimiento del proyecto y contener al mismo tiempo explicaciones, ejemplos o decisiones editoriales mantenidas por una persona.

IASI debe permitir esa combinación sin convertir la regeneración en una trituradora de trabajo humano.

Una estrategia útil separa:

```
contenido derivable
+
includes de autor
```

Al regenerar:

- IASI actualiza lo derivable;
- conserva los includes de autor;
- detecta si las dependencias del contenido de autor han cambiado;
- marca ese contenido como **affected** cuando necesita revisión;
- no lo reescribe automáticamente salvo que exista autorización explícita.

Este patrón generaliza más allá de la documentación.

Permite trabajar con cualquier artefacto parcialmente derivable manteniendo una frontera clara entre **materialización automática** y **aportación profesional deliberada**.



## Referencias

Las referencias bibliográficas utilizadas en este volumen se incorporarán a medida que los capítulos requieran fuentes externas. Los elementos normativos propios de IASI se documentan, cuando procede, junto con sus fuentes y su justificación.



# Licencias

El proyecto **iasi** contiene materiales de distinta naturaleza. Cada tipo de material se distribuye bajo una licencia específica.

Copyright © 2026 Javier G. Grandez.

## Código fuente

El código fuente, los scripts, las automatizaciones, las utilidades y los ejemplos ejecutables se distribuyen bajo la **Licencia MIT**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- scripts escritos en R;
- filtros, extensiones y utilidades;
- archivos de automatización;
- herramientas de construcción y publicación;
- fragmentos y ejemplos de código reutilizables.

La Licencia MIT permite utilizar, copiar, modificar, integrar, publicar, distribuir, sublicenciar y vender el código, siempre que se conserve el aviso de copyright y el texto de la licencia.

El texto completo está disponible en:

[Licencia MIT](#)

Identificador SPDX: MIT.

## Contenido editorial

Los textos, capítulos, diagramas, tablas e imágenes originales de la obra **Ingeniería aumentada con sistemas inteligentes** se distribuyen bajo la licencia **Creative Commons Attribution 4.0 International**, salvo que se indique expresamente otra licencia.

Esto incluye, entre otros:

- los documentos y capítulos escritos en formato `.qmd`;
- los diagramas, tablas e ilustraciones originales;
- las versiones HTML, PDF y otros formatos generados a partir de la obra.

Esta licencia permite compartir y adaptar el contenido, incluso con fines comerciales, siempre que se reconozca adecuadamente la autoría, se proporcione una referencia a la licencia y se indiquen los cambios realizados.

El texto legal completo está disponible en:

[Creative Commons Attribution 4.0 International](#)

Identificador SPDX: CC-BY-4.0.

## Atribución

Para reutilizar el contenido editorial, se recomienda utilizar una atribución semejante a la siguiente:

Javier G. Grandez, *Ingeniería aumentada con sistemas inteligentes*, 2026. Licencia CC BY 4.0.

Cuando el contenido haya sido modificado, deberá indicarse de forma razonable que se han realizado cambios.

## Materiales de terceros

Los materiales pertenecientes a terceros conservan sus respectivas licencias, derechos de autor y condiciones de uso.

Su inclusión o referencia en este proyecto no implica que estén cubiertos por la Licencia MIT ni por la licencia CC BY 4.0.

Cuando corresponda, la autoría, la procedencia y la licencia aplicable se indicarán junto al material o en su referencia bibliográfica.

## Alcance

La Licencia MIT se aplica al **código**.

La licencia CC BY 4.0 se aplica al **contenido editorial original**.

Cuando un archivo incluya una indicación de licencia específica, esa indicación prevalecerá para dicho archivo.